BeltFlow | Complete Implementation Plan

Master-led classes. Persatuan-approved network. Clear student progression.

PRODUCT SPECIFICATION + IMPLEMENTATION ROADMAP + ANTIGRAVITY HANDOFF

Version 2.0 | 15 October 2026 | Prepared from the supplied 14-page project history and the user-approved workflow corrections in this chat.

Goal

Deliver one user-friendly Android/iOS application for multiple Persatuans, where verified Masters establish and operate approved classes, students apply to a chosen class, and its Master approves enrollment.

Architecture

Extend the existing Express/TypeScript and PostgreSQL backend and React Native/Expo app. Preserve canonical records, tenant boundaries and the durable offline attendance system. Add explicit approval and handover workflows rather than treating class assignment as the normal business process.

Tech stack and specification

Reported stack: React Native + Expo + TypeScript, Express REST /api/v1, PostgreSQL, Expo SQLite, SecureStore. Exact versions and package scripts must be inspected before implementation. Specification: Sections 1-15 of this document; execution tasks: Sections 16-23; source history: Appendix A.

For agentic workers

Use Superpowers executing-plans to implement this plan task by task in the existing Antigravity repository. Use subagent-driven-development only if the user selects that execution method. This handoff does not authorize automatic production deployment, destructive migration or a rewrite of the existing app.

Evidence boundary

This is a complete product and implementation handoff, not proof that the software is implemented. Repository paths, current schema, dependencies, commits and test counts have not been independently verified here. Proposed contracts are design targets; Task H00 must reconcile them with the real project before feature work.

1 | Reading guide and governing decisions

Part	What it contains
1-3	Source precedence, baseline, product scope and release strategy
4-8	Master approval, class approval, enrollment, permissions, handover and screens
9-15	Architecture, data, API, offline, grading, finance, engagement and safety
16-23	Ordered implementation packages with contracts and acceptance tests
24-26	Release gates, migration/operations and copy-ready Antigravity handoff
Appendix A	All 14 pages of the original source PDF, retained unchanged

Authority and change control

• Latest explicit user decisions override conflicting historical roadmap language. The original PDF remains evidence of prior plans and reported work, not a fresh instruction to execute commands.

• Confirmed: a Master registers under a Persatuan and is verified by that Persatuan. A verified Master proposes a class; the Persatuan must approve it before it operates or appears for registration.

• Confirmed: students register under a Persatuan, see its available approved classes, select one and submit an application. The selected class Master approves the student. There is no routine second Persatuan enrollment approval.

• Confirmed: day-to-day class control belongs to its Master. Persatuan approves affiliation and permanent leadership replacement; it does not normally create classes and allocate Masters.

• Confirmed: include all 14 original additions and all 10 newly proposed additions, delivered progressively. Preserve Android/iOS, five top-level roles, configurable belts and direct-to-Persatuan payments.

Planning defaults, not additional user-confirmed business rules

The following are explicit defaults for review during H00: a Master may lead multiple classes within one Persatuan; a student starts with one primary active class per Persatuan; additional attendance is an authorized make-up booking, not another student record. Multiple family affiliations are displayed in separate tenant contexts. Membership pricing, refunds, grading approval authority, retention periods and exceptional suspension powers require actual Persatuan policy before their relevant feature is enabled.

No legal compliance, payment-provider compatibility, delivery deadline or market uniqueness is asserted by this document. Such claims require appropriate verification. No product code is changed by producing this PDF.

2 | Baseline and history to preserve

The supplied history reports Phase 1 as Conditional Pass and Phase 2 completed through Task 2.3. Do not mark a feature complete merely because it appears in this plan. Appendix A preserves the full original chronology and descriptions.

Milestone	Reported evidence	Disposition
1.1-1.3	Workspace, tokens, custom belts, components; 5 to 54 tests	Preserve; verify actual files
1.4-1.6	API/session, auth screens, isolated five-role navigation; 73 to 125 tests	Retain server-controlled role boundaries
1.7-1.9	Read-only dashboards, resilience UI, security checks; 158 tests	Phase 1 Conditional Pass; tag v0.1.0-phase1; 64d7b1c
Phase 2 gate	61 live HTTP assertions; 158 mobile tests; 5c55c55	Native launch evidence absent
2.1	SQLite repositories/migrations; 173 tests; a953b27	Preserve queue identities and cache restrictions
2.2	Offline Master attendance; 191 tests; 02b6a90	Preserve corrections, quarantine and account switching
2.3	Sync API and worker; 216 tests; 94f82ed	Verify atomicity, concurrency and authorization
2.4 / 2.5 / 2.6	QR / persistent belts / certification	Planned; no completion evidence supplied

Open defects and missing evidence

• CERT-001: welcome-screen public certificate verification conflicts with a JWT-protected API. Provide a narrow public-safe verification contract.

• NATIVE-001 and NATIVE-002: no verified Android or iOS native launch in the supplied history. Metro exports are not runtime proof.

• Sync: verify attendance write plus processed mutation record commit atomically; reject stale concurrent updates safely; reauthorize queued mutations.

• Local privacy: verify backups, account switch scrubbing, restricted cache retention and queued-change recovery on actual devices.

• Belt configuration: reported in-memory modeling is not persistent tenant-owned configuration.

• New business-model gap: inspect whether class creation, affiliation approval, Master verification and student applications exist. Do not infer their absence solely from the phrase assigned classes.

3 | Product identity and delivery strategy

Distinctive product direction

BeltFlow should connect a Master-run class network with a visible, verified student learning journey. Its identity is the combination of local class autonomy, Persatuan standards, offline attendance, flexible belt curricula and clear family communication. Avoid crowded menus and unsupported claims that competitors lack these capabilities.

Release	Scope	Exit condition
R0: trusted baseline	Inventory, correct authority model, protect existing data, native smoke tests	Known baseline and tested migration approach
R1: core academy	Master/class/student approvals, QR/offline attendance, scheduling, membership, safety, persistent belts, grading, payments, receipts, notices, onboarding	Complete native end-to-end academy journey and financial verification
R2: teaching and retention	Readiness passport depth, learning, performance, lesson plans, family hub, make-ups, pauses, transfers, follow-ups, rewards and video	Consent-aware workflows and usable daily operations
R3: events and commerce	Tournaments, judging, merchandise, deeper analytics and full messaging	Reconciled transactions, controlled scoring and permissions
R4: assisted operations	Grounded role-aware AI and advanced assistance	Tenant-safe evaluations, human approval boundaries and usage controls

Preserve the six-phase framework

P1 foundation; P2 academy operations; P3 grading and learning; P4 finance and commerce; P5 tournaments and engagement; P6 AI and release hardening. Releases span phases: R1 includes basic P3 grading, P4 verified payments and P5 notices. Native testing, accessibility, localization and privacy apply from the start, not only in P6.

User-friendly operating rules

• Each role lands on one useful Today screen. Reveal tools by permission and enabled module, not by showing every future feature.

• Use explicit states: Awaiting Persatuan approval, Awaiting Master approval, Saved on this device, Pending sync, Payment pending and Confirmed payment.

• Keep important actions within a short flow with a review step, clear result and recovery route. Never imply a local save is server approval.

• Preserve navy #0D2B47 and amber #F2941C branding; use high-contrast text, large touch targets, screen-reader labels, scalable text and localized dates/currency.

• Measure pilot task completion, enrollment abandonment, attendance correction rate, sync recovery and staff time saved. Establish targets from the pilot rather than inventing business uplift.

4 | Master and class approval workflows

Master registration and verification

Account creation and verified teaching authority are separate. Registration captures the chosen Persatuan and policy-required verification material through restricted channels. Before approval, the user sees only application status, requested corrections and support; no student roster or class operations are available.

Master application states: DRAFT -> SUBMITTED -> NEEDS_INFORMATION -> SUBMITTED -> APPROVED or REJECTED. WITHDRAWN is allowed before a final decision. Approved Master affiliation is a separate durable record with ACTIVE, SUSPENDED or ENDED status. Suspension powers must be configured under the approved governance policy, not assumed to permit routine class takeover.

Master-created class

Only an ACTIVE verified Master affiliation may submit a class. The proposed class contains Persatuan, accountable Master, venue, contact channel, timetable, program, age/level eligibility, capacity and registration information. The backend derives applicant identity and validates the Persatuan; it does not trust arbitrary owner IDs from the phone.

Class approval states: DRAFT -> SUBMITTED -> NEEDS_INFORMATION -> SUBMITTED -> APPROVED or REJECTED. Store operational status separately: NOT_STARTED, ACTIVE, PAUSED or CLOSED. Only an approved class can become ACTIVE. Enrollment availability is separately OPEN, WAITLIST or CLOSED; an approved class is not necessarily accepting new students.

Approval integrity

• A review records reviewer, timestamp, application version and reason. Approve exactly the reviewed version; concurrent edits require a fresh review.

• A Master cannot self-approve through any role, endpoint or crafted request. Decide dual-role conflict handling if the existing model supports multiple roles.

• Persatuan admin reviews only applications in its own organization. Platform Super Admin cannot substitute for this operational decision.

• Initial approval must precede live classes, attendance and student enrollment. Draft preparation is allowed, but it creates no active students or sessions.

• Routine schedule and lesson changes remain under the Master. A change of Persatuan affiliation or permanent Master is never a simple editable field; use its explicit governed workflow.

Public discovery defaults

Show only approved, active classes within the selected Persatuan. Publish approved venue details, program, schedule, eligibility, published fees and availability. Do not publish student names, private Master contacts, rosters, internal notes or verification documents. Waitlist is a distinct opt-in application route once implemented.

5 | Student enrollment and family identity

Enrollment journey

Choose Persatuan -> browse its approved available classes -> inspect class details -> choose class -> student or guardian completes application -> submit -> selected class Master reviews -> approve -> active membership and class enrollment. Until approval, show application status rather than class-only records.

Application states: DRAFT, SUBMITTED, NEEDS_INFORMATION, WAITLISTED, APPROVED, REJECTED, WITHDRAWN. WAITLISTED does not equal an active student. Rejection and withdrawal preserve an appropriately restricted decision history under the retention policy.

Atomic approval

approveEnrollment(applicationId, expectedVersion, idempotencyKey) validates the current Master assignment, active class, approved affiliation, eligibility, consent requirements and capacity. In one transaction it links or creates the canonical student, creates the enrollment, updates the application and records an audit/outbox event. A retried request must not create duplicate students, memberships or charges.

Identity and parent links

• A child may have a student record without an independent login. Guardians use validated links; supplying a child name or knowing an application ID does not grant access.

• Use tenant-scoped identity matching and a review queue for possible duplicates. Do not automatically merge children by name, phone number or shared family email.

• Store account, person/student, guardian link, Persatuan membership and class enrollment as distinct concepts. Approval attaches relationships rather than cloning a student on each move.

• Default to one primary active class per Persatuan; retain enrollment history with effective dates. Support future policy-driven multi-class enrollment without changing student identity.

• Keep enrollment approval separate from payment status. Activation is governed by the approved enrollment rule; financial eligibility for particular services is an explicit policy, never an accidental side effect of a payment webhook.

What the Master sees

A pending-applications inbox displays only information needed to assess their class applications. The Master can request missing details, approve or decline with a reason. A capacity race produces a clear full-class result, not a silent overbooking. Families receive status updates without exposing other applicants.

Trial conversion

A trial uses consent and limited attendance access. Conversion links the trial applicant to the same student and enrollment process. It still requires class Master approval; it must not create a second student or bypass class approval.

6 | Roles, permissions and ownership

Keep five top-level roles. Treat role, Persatuan affiliation, active class leadership and delegated permission as separate authorization inputs. Every server action validates tenant, subject, scope, status and effective dates. UI visibility alone is not security.

Role	Allowed scope	Boundary
SUPER_ADMIN	Provision Persatuans, platform configuration, technical health and redacted audit metadata	No student, attendance, fees, grading, child media or safety operational records
ADMIN_PERSATUAN	Verify Masters; approve classes; configure curricula; approve permanent handovers; authorized organizational reports	No default attendance editing, teaching impersonation or unrestricted class operations
MASTER	Create class proposals; operate classes they currently lead; approve applicants; teach, assess and record attendance	No self-approval, cross-class access or Persatuan-wide curriculum editing
PARENT	Validated children; applications, schedules, authorized dues, notices, consent and progress	No access via guessed child ID; separate tenant context
STUDENT	Own approved enrollment, schedule, progress, pass and authorized learning	No peer records, grading decisions or class management

Proposed capability examples

master_application.review; class_application.review; class.manage; enrollment.review; attendance.write; assessment.write; curriculum.manage; grading.finalize; finance.collect; finance.refund; safety.read; safety.write; handover.approve; judging.score; inventory.manage. These are proposed names to map to existing authorization helpers.

Granularity and financial access

Permissions are action-specific. A Master may review own-class dues without receiving unrestricted merchant credentials or unrelated family financial details. Refunds and final grading approval require explicit delegated authority. Admin reports may use approved aggregates without granting raw student-record access. Verification documents and safety records are not ordinary profile fields.

Revocation and delegation

A revoked or ended assignment blocks new server writes immediately. Short-lived substitute/judge permissions have scope and expiry. Offline devices cannot learn revocations instantly; cached work stays provisional and must be reauthorized before sync. No client may extend its own permission expiry.

7 | Class continuity, handover and transfers

Permanent Master replacement

A handover is a leadership change, not a class deletion or transfer of student identity. The outgoing Master can request it; the Persatuan can initiate a documented replacement when the outgoing Master is unavailable. The incoming Master must have active verified affiliation in the same Persatuan and acknowledge the handover.

States: REQUESTED -> INCOMING_ACCEPTED -> APPROVED -> SCHEDULED -> COMPLETED; alternatively REJECTED or CANCELLED before execution. Store initiator, approver, outgoing/incoming IDs, reason, effective time, expected class version and a pending-work summary.

• At the effective time, change leadership in one transaction and prevent overlapping active primary leadership. Revoke outgoing operational permissions and grant incoming permissions.

• Preserve immutable authorship on historical attendance, assessments and receipts. The incoming Master sees permitted class history, not the outgoing Master account or credentials.

• List pending enrollment applications, unresolved sync conflicts, open incidents with restricted disclosure, scheduled classes and permitted financial tasks. Do not transfer private inboxes wholesale.

• Queued edits from the outgoing Master after revocation do not replay under the incoming Master identity. Quarantine them for a controlled, auditable resolution workflow.

• After completion, notify families of the approved change. A notification failure does not roll back leadership; it retries through the outbox.

Temporary substitutes

A substitute receives time-bounded session permissions under academy policy. The primary class Master stays unchanged. Substitute access does not include historic private messages, unrelated safety details, merchant configuration or permanent leadership changes.

Student class transfers

Within a Persatuan, request the target class, verify eligibility/capacity, obtain authorized release and target-Master approval, then end the old enrollment and begin the new one at an effective date. Preserve student identity and learning history. Existing balances are not silently reassigned or erased.

Cross-Persatuan transfer is a distinct later workflow with explicit receiving acceptance and consent for the specific records shared. Do not copy tenant-private incidents, financial data or videos by default. A receiving curriculum determines equivalence; the system must not invent belt equivalence.

8 | App navigation and screen structure

Experience	Primary screens	Focused actions
Before approval	Welcome; account; Persatuan selection; application; status; correction request	Apply, supply missing details, see who reviews next
Master	Today; My Classes; Applicants; Students; Teaching; More	Create class, start attendance, review applicants, resolve pending sync
Persatuan admin	Overview; Approvals; Classes; Masters; Settings	Verify Master, review class, approve handover, manage curriculum
Parent	Family Today; Classes; Progress; Payments; Inbox	Choose child, apply, acknowledge notice, pay correct Persatuan
Student	Today; My Progress; Learn; Digital Pass; Inbox	See next class, understand next skill, access own pass
Super Admin	Organizations; Platform Health; Provisioning; Audit Metadata	Manage platform without entering academy operations

Screen modules to include

Class detail: approval badge, venue, published timetable, eligibility, fee summary and availability. Class workspace: roster, sessions, applications, lessons, scoped finances and settings. Student profile: enrollment, attendance, progress, membership and separately protected safety fields. Approvals inbox: reason, evidence, version, reviewer action and result.

Reusable UX states

• Every list supports loading, empty, unavailable, permission denied and retry states. Important drafts survive accidental navigation where permitted.

• Every mutation shows submitting, success or a meaningful failure. Disable repeated taps without relying on that as the only duplicate prevention.

• Connectivity banners distinguish no internet from server unreachable. Pending sync and conflict counts come from the durable queue.

• Use a child selector in the family hub and a visible Persatuan label. Never combine different merchants into one payment checkout.

• Display pending approval as a guided status page with the next action; do not expose an empty full-role dashboard.

Localization and accessibility

All new strings use localization keys from their first implementation. English, Malay and Tamil are the target languages. Test long translated labels, Tamil font coverage, dynamic font size, screen readers, contrast and touch targets. User-authored curriculum content retains its original language unless an authorized translation exists; machine translation must be labeled.

9 | System architecture and module boundaries

Preserve the existing backend and PostgreSQL source of truth. The phone holds authorized caches and a durable attendance queue; it is not an approval authority or financial ledger. Reuse existing authentication, classes, students, attendance, billing, grading, certificates, coaches, tournaments, skills, notices and audit domains.

Boundary	Responsibility	Dependencies
Identity and authorization	Account, role, affiliation, guardian relationships, effective scopes	Existing auth and organization model
Academy network	Master verification, class proposal/review, leadership history	Authorization and audit
Enrollment and scheduling	Applications, class membership, capacity, sessions, make-ups, pauses	Approved classes and student identity
Attendance and sync	Offline cache, mutations, versions, conflict resolution, QR resolution	Current enrollment and session authorization
Curriculum and learning	Versioned belts, assessments, readiness, lessons, media and rewards	Tenant configuration and verified attendance
Finance	Bills, provider requests, webhook verification, receipts, refunds and ledger	Tenant merchant mapping and membership
Engagement and commerce	Notices, messaging, tournaments, stock, orders and reports	Scoped canonical records and ledger
Cross-cutting services	Audit, outbox, consent, uploads, retention, monitoring and AI retrieval	Least-privilege server interfaces

Proposed file map, subject to H00

Follow the repository layout first. If no matching module exists, proposed logical roots are server/src/modules/<module>/ and mobile/src/features/<module>/. A server module separates routes, request schema, authorization policy, service and repository; mobile separates screens, hooks, API adapter and local-store adapter. Use existing migration and test directories rather than creating parallel conventions.

Each implementation task below names its logical module. H00 must create an exact file map linking that name to real paths, existing interfaces, tests and package scripts. Do not fabricate line numbers or assume the illustrative roots already exist.

Integration rules

State changes plus their audit/outbox events commit together where applicable. Workers are retryable and idempotent. Version mutable resources; enforce optimistic updates in SQL, not just in client checks. Avoid unnecessary service splitting: well-defined modules in the existing backend are sufficient.

10 | Data model and migration contract

Names below are proposed additions or extensions, not a claim about deployed tables. Reuse existing equivalents after inspecting constraints and migration history. Every tenant-owned reference must enforce same-tenant consistency, preferably with composite keys/foreign keys as well as service checks.

Entity / extension	Key relationships and constraints
master_applications; master_affiliations	Account + Persatuan; reviewed version, reviewer/reason; affiliation status independent from login
class_applications; classes	Persatuan + applicant Master; approval state separate from operational/enrollment state
class_leaderships; class_handovers	Class + Master + effective period; no overlapping active primary leader; full audit
enrollment_applications; class_enrollments	Student/person + class + Persatuan; application status and version; primary membership history
guardian_links; consent_records	Verified relationship, scope, evidence and effective dates; no child-ID-based access
schedule_exceptions; bookings; membership_events	Session capacity; atomic bookings; effective pause/resume dates and policy snapshots
belt_system_versions; ranks; criteria	Tenant-owned curriculum; stable rank IDs; versioned labels, sequence and eligibility
assessments; scores; lesson_plans	Template version, skill evidence, author, class/student, moderation/finalization status
payments / ledger extensions	Provider/merchant references, idempotent events, bill allocations, refunds and immutable entries
notification_outbox; acknowledgements	Tenant-scoped recipients; delivery retry; acknowledgement distinct from legal consent
safety/media/messaging extensions	Purpose-limited access, retention, consent and access logs
imports; transfers; inventory; tournament extensions	Dry-run imports, record mapping, stock movements, score locks and appeals

Migration safety

Use expand -> backfill -> validate -> switch reads/writes -> contract only after approval. Do not auto-approve every legacy class or Master merely to populate a new status field. Produce a legacy review report, identify verified documentary evidence, and have authorized Persatuan reviewers confirm the migration mapping before enforcement.

Preserve existing IDs, ownership history, fees and certificates. Record provenance for migrated approvals separately from live approvals. Test with a sanitized production-shaped copy; back up and rehearse restore. Add indexes for tenant/status inboxes, class/session rosters, effective leadership and unique provider events.

11 | API contracts and application events

Proposed routes below are relative to /api/v1. Extend an existing equivalent rather than publish a duplicate. Identifiers are opaque strings; timestamps are UTC ISO 8601; localized display happens on the client. tenantId and actorId are derived/validated server-side.

Interface	Behavior
POST /master-applications; POST /master-applications/:id/review	Create application; approve/reject/request information using expectedVersion and reason
POST /class-applications; POST /class-applications/:id/review	Verified Master proposes; same-Persatuan reviewer decides reviewed version
GET /persatuans/:id/available-classes	Approved active classes with OPEN availability; distinct optional waitlist filter; public-safe projection
POST /enrollment-applications; POST /enrollment-applications/:id/review	Apply to selected class; current class Master approves atomically with capacity check
POST /classes/:id/handovers; POST /handovers/:id/approve	Controlled replacement with incoming acceptance and effective time
POST /attendance/sync	Preserve existing per-mutation identity/version/status contract and batch limit 25
GET /students/:id/pass; POST /attendance/resolve-pass	Authorized pass issuance/resolution; strict class/session/tenant validation
POST /payments; POST /payments/webhooks/:provider	Server-owned merchant/bill allocation; verified provider event processing
GET /certificates/verify/:token	Minimal public verification projection; rate-limited; no student directory
POST /notices/:id/acknowledgements	Record authorized recipient acknowledgement idempotently

Common mutations

Input: resource ID, expectedVersion, idempotencyKey and action-specific validated fields. Success returns resource ID, state, version and timestamp. Same key with different payload is rejected. Client recovery can query status after timeout; never retry by creating a new identity for the same logical approval or payment.

Errors: 400 validation; 401 unauthenticated; 403 unauthorized; 404 inaccessible/not-found resources without enumeration; 409 stale version, invalid transition, duplicate key mismatch or full capacity; 429 rate limit. Include a stable error code and localized client mapping, not raw SQL or private details.

Events

MasterApproved, ClassApproved, EnrollmentApproved, LeadershipChanged, AttendanceSynced, GradingFinalized, PaymentConfirmed and NoticePublished create scoped outbox records. Consumers deduplicate events. Notifications and analytics do not decide authoritative approval, grading or settlement states.

12 | Offline attendance and QR specification

Keep the reported implementation

Preserve SQLite migrations, session/roster repositories, attendance_mutation_queue, sync_conflicts and sync_metadata. SecureStore holds session credentials; do not put bearer JWTs, bank credentials, national IDs or full safety profiles in the roster database. Keep reported batch size 25 unless performance evidence justifies a reviewed change.

Lifecycle

Online authorization -> download assigned/led roster and session -> offline attendance save -> Pending sync -> server reauthorization and version check -> commit attendance and idempotency ledger together -> acknowledged or explicit conflict/quarantine. Account change, expired authority, closed sessions and revoked leadership require clear recovery paths.

QR pass

Issue an opaque non-enumerable online credential or a signed, expiring minimal offline-verifiable credential after the chosen design is audited. Never encode JWT, phone, address, NRIC/MyKid, payment details or a readable student dossier. A pass identifies a record; it does not prove the presenter is the student.

• Master selects an authorized active session before scanning. Resolve against the current authorized roster; parent sees passes only for verified linked children.

• The camera handles denied permission, poor lighting, flash support, invalid QR, expired pass, wrong class/Persatuan and duplicate scans. Provide manual roster attendance.

• Offline verification must use a trusted public key/authorized cache and a declared validity policy. Do not ship a shared signing secret in the mobile app.

• Offline revocation is not real-time. Mark records provisional and verify current authority, enrollment and pass policy at sync.

• Choose and document credential validity and offline authorization windows before shipping; they are security/product settings, not arbitrary constants supplied by this plan.

Required conflict behavior

Treat simultaneous stale edits as conflicts, not last-write-wins guesses. Display who changed the record when disclosure is authorized and offer a reviewed correction. Quarantined changes must be visible to an authorized recovery workflow; do not silently discard them or replay under another account.

Native acceptance

Run a 50-student offline session, kill and reopen the app, reconnect, repeat sync, correct records and revoke leadership during disconnection. Test actual Android and iOS camera behavior, secure storage, SQLite persistence and account switching. Background execution is best-effort; foreground recovery is mandatory.

13 | Grading, learning and student development

Tenant-specific belts and history

Persatuan controls belt count, names, colors, sequence, explicit Dan/instructor metadata, fees, minimum sessions/months and assessment rules. An optional eight-rank template is not a global default truth. Version rules and preserve rank names, certificate display and eligibility evidence used at the time of an award.

Readiness passport and assessment

Show required skills, verified attendance, time at rank, instructor sign-offs and remaining steps. Record mastery evidence and rubric versions. The Master reviews readiness; the appropriate delegated authority finalizes grading. Neither attendance alone nor AI automatically awards a belt. Appeal/correction retains the original audit trail.

Performance intelligence and rewards

Use configurable rubrics such as power, speed, style, skill and variety only where the curriculum defines them. Display progress over comparable assessments; do not imply scores from different rubrics are equivalent. Rewards come from verified events, can be revoked with reason and avoid public child rankings by default.

Learning library and lesson handover

Persatuan-approved learning modules reuse syllabus skills. Track lesson progress and quizzes separately from formal grading. A Master plans session objectives, drills and equipment, records coverage, and provides limited substitute handover. Content ownership and publishing permissions are explicit.

Video training feedback

Require scoped media consent, authorized uploads, file/size validation, private storage and expiring access URLs. Link comments/annotations to the student and lesson. Withdrawal blocks future sharing and triggers the defined retention/deletion process. No public child gallery or automatic biometric assessment is assumed.

Certificates

Issue only from finalized grading. Store a historical display snapshot and a revocation/correction mechanism. Public verification returns minimal validity details and carefully limited identity display; it must not expose contact details, attendance, dues, full birth date or a searchable student index. Resolve CERT-001 with contract and abuse tests.

14 | Finance, membership and commerce

Direct merchant settlement

Each Persatuan receives money through its own approved merchant account. BeltFlow does not pool settlement. Select and validate a provider that actually supports the required FPX/DuitNow arrangement; this document does not claim a specific provider or integration is already approved.

A payment request is bound server-side to tenant, merchant, payer, bills, amount and currency. The app return link is only a navigation signal. Confirm money only after verified provider callbacks/status reconciliation. Keep manual proof uploads clearly separate from gateway-confirmed payments.

One auditable ledger

Tuition, membership renewal, grading, events and merchandise use one ledger and explicit allocations. Store currency amounts using integer minor units or an exact decimal representation. Never recompute historic receipts from editable current prices. Duplicate, delayed and out-of-order callbacks must not double-credit or regress final state.

Membership pauses, renewals and refunds

Membership status, class enrollment and payment state are related but distinct. A pause records effective dates, approved policy and return reminder. Freeze billing or extend expiry only when the policy says so. Renewal creates an auditable event. Refunds require scoped authority, provider confirmation and compensating ledger entries, not deletion of the original payment.

Family payment experience

Show a child-specific breakdown and the receiving Persatuan before confirmation. A family may view multiple affiliations but checks out separately for different merchants. Permission to pay for a child does not grant access to unrelated guardians or another tenant.

Equipment and merchandise

Products, stock movements, reservations, orders and refunds link to the ledger. Reserve stock atomically with expiry; handle failed and late payments without negative inventory. Staff inventory authority is scoped, not a new global role. Equipment tracking may support class-owned loan items separately from stock sold to families.

Acceptance and operational evidence

Use the chosen gateway sandbox for signature validation, merchant mismatch, amount mismatch, duplicate event, timeout, late success, refund and reconciliation scenarios. Verify receipt accuracy and store no banking secrets on the device. Production merchant onboarding and actual payment activation are separate authorized operations.

15 | Engagement, safety, growth and AI

Scheduling, make-ups and follow-up

Support recurring schedules, exceptions, venue changes and substitute sessions with clear local time display. Make-up requests validate age/level, capacity and membership policy, then require the authorized approval. Waitlist promotion is atomic. Retention follow-ups use transparent configurable rules for repeated absence, missed trials or expiry; a Master records the contact outcome without labeling a student with unexplained AI risk scores.

Notices and messaging

Ship essential class notices and acknowledgement early. Delivery, read and acknowledged are distinct states. Full parent-Master messaging adds scoped threads, reporting, moderation and retention; a new class Master does not automatically inherit private conversations. Child messaging restrictions must be defined before enablement.

Safety

Emergency contacts, pickup permissions, incident logs and health alerts are separately protected. Only users with a current need-to-know see them. Keep sensitive details out of lock-screen notifications and routine analytics. Offline safety access, if needed, requires its own minimum-data, encryption, expiry and revocation design rather than reuse of the attendance cache.

Tournaments and electronic judging

Reuse event registration and competition history. Add categories, eligibility, brackets, judge assignments, scorecards, score locks, tie rules, appeals and medal results. Select a sport/Persatuan scoring ruleset explicitly. Authorized judges cannot silently edit locked results; corrections and appeals are auditable. Competition connectivity failures require a rehearsed fallback before live use.

Growth analytics

Define active student, retention, trial conversion, utilization, billed, collected and outstanding before charts are built. Apply tenant/class scope to queries and exports. Use aggregate reports for authorized admin oversight; aggregate access must not become a route to unrestricted student data.

AI assistant

Start with questions answered from approved curriculum, help material and authorized personal records. Apply authorization before retrieval; never give Super Admin child-level data through AI. Treat uploaded content as untrusted. Cite sources, admit missing evidence, limit costs and log access without unnecessary sensitive text. No autonomous grading, medical advice, financial approval, student rejection or direct messaging. Any later write action requires a separately designed permission and human confirmation flow.

16 | Implementation method and review focus

Global constraints

• One Android/iOS app; preserve existing Express/PostgreSQL backend and canonical records. No new customer-facing web app.

• Five top-level roles. Master and class approval precede operation; class Master approves students.

• Masters control their own classes; Persatuan affiliation approval is not routine operational takeover.

• Tenant isolation, guardian validation and effective class scope are enforced server-side.

• Offline attendance is provisional until authorized server sync. Financial success requires provider verification.

• Belt rules and historical artifacts are tenant-specific and versioned. Localization, accessibility and native checks accompany feature work.

• All historical completion claims remain reported until independently verified. No task completion without actual evidence.

Repository mapping before implementation

H00 writes an exact file/command map for each logical module below. For example, academy-network maps its route/controller/service/repository, mobile approval screens, migrations and contract tests to actual paths. Record existing interface signatures before changing them. Where a route already exists, adapt it and its consumers rather than adding a second system.

Per-task test cycle

Each task uses the checkbox sequence printed on its card. Use the repository package manager and actual scripts discovered in H00. A failing test must fail for the intended missing behavior; a passing suite must include the new assertions. Run relevant typecheck/lint, contract/database tests and native tests when the task touches native behavior. Record exact commands and results in the completion report; do not invent universal script names here.

Five high-risk review inputs and owning tasks

Condition	Expected result / task
Two approvals for the final class place	One enrollment succeeds; other gets capacity conflict; no duplicates. H05.
Approval arrives after applicant edits	Reject stale version and require fresh review. H03/H04.
Master leaves while device is offline	Old queued writes quarantined; no replay as replacement. H07/H08.
Delayed provider success after timeout	Reconcile once to the correct merchant/bill; no duplicate receipt. H13.
Duplicate child names or multilingual long text	No automatic identity merge; readable accessible application. H05/H18/H28.

17 | Tasks H00-H03: baseline and authority

H00 | Repository discovery and compatibility map

Depends on: none. Proposed module: handoff-audit.

Contract: produce baseline-report, exact-file-map, actual-command-map and decision-register

Deliver: Inspect branch/status, supplied commits, migrations, routes, permissions, package versions and tests. Read local AGENTS instructions. Inventory all planned features against actual code; classify complete/partial/missing/unverified. Resolve policy defaults with existing evidence; only ask focused questions where implementation depends on them.

Acceptance tests: baseline_report_separates_reported_from_verified; every module maps to real paths; no production data mutated.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H01 | Native and sync readiness

Depends on: H00. Proposed module: attendance/auth.

Contract: preserve sync(batch<=25) and existing queue semantics

Deliver: Run actual Android/iOS smoke paths; repair necessary runtime issues. Verify transactional attendance/idempotency writes, race-safe versions, crash recovery, account switch and revoked-authority replay. Track unavailable device evidence as a blocker, never a pass.

Acceptance tests: sync_commit_and_ledger_are_atomic; stale_parallel_mutations_conflict; replay_same_id_once; account_switch_no_roster_leak; native_launch_evidence.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H02 | Authority relationships and legacy migration

Depends on: H00. Proposed module: academy-network/policy.

Contract: authorize(actor, capability, resource, effectiveAt) -> decision

Deliver: Map affiliations, class leadership, approvals and enrollments. Implement additive constraints and a reviewed legacy backfill path; preserve original IDs and history. Distinguish pending registration from verified operational authority.

Acceptance tests: cross_tenant_relationship_rejected; overlapping_leadership_rejected; pending_master_cannot_operate; legacy_backfill_does_not_silently_approve.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H03 | Master verification

Depends on: H02. Proposed module: master-applications.

Contract: submitMasterApplication(input); reviewMasterApplication(id, decision, expectedVersion, reason)

Deliver: Implement application/status/correction screens and Persatuan review inbox. Approval activates affiliation with audit/outbox event. Keep verification materials restricted; rejection never deletes unrelated account history.

Acceptance tests: same_tenant_admin_only; self_approval_rejected; stale_review_rejected; duplicate_approval_idempotent; pending_account_has_no_roster.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

18 | Tasks H04-H07: class and student lifecycle

H04 | Master-created class approval

Depends on: H03. Proposed module: class-applications.

Contract: submitClassProposal(input); reviewClassProposal(id, decision, expectedVersion); listAvailableClasses(persatuanId)

Deliver: Add Master-owned proposal and Persatuan review. Separate approval, operational and enrollment states. Only approved active open classes enter public-safe discovery.

Acceptance tests: unverified_master_cannot_submit; unapproved_class_cannot_operate; own_tenant_review_only; stale_proposal_review_rejected; unavailable_classes_hidden.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H05 | Student application and Master approval

Depends on: H04. Proposed module: enrollment.

Contract: applyToClass(input); approveEnrollment(id, expectedVersion, idempotencyKey)

Deliver: Build student/guardian application, validated guardian relationships, Master inbox and atomic approval. Use canonical identity and capacity locking; do not add a second routine admin approval.

Acceptance tests: selected_class_master_only; last_place_race_one_winner; retry_no_duplicate_student; tenant_mismatch_rejected; same_name_children_not_merged; pending_applicant_no_roster.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H06 | Master class workspace and basic rosters

Depends on: H05. Proposed module: class-workspace.

Contract: getClassWorkspace(classId); listClassStudents(classId); updateClassSettings(id, version, patch)

Deliver: Connect approved enrollments to existing attendance roster queries. Build Master Today, My Classes, pending applications and class settings. No class assignment wizard as the standard creation path.

Acceptance tests: own_led_classes_only; new_approved_student_enters_roster; pending_student_excluded; former_master_access_denied; Today_counts_match_authorized_queries.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H07 | Permanent handover and temporary substitution

Depends on: H06/H01. Proposed module: leadership.

Contract: requestHandover(input); acceptHandover(id); approveHandover(id, version); executeDueHandover(id)

Deliver: Implement incoming acceptance, Persatuan approval and effective-time atomic leadership change. Preserve class identity and authorship. Add separate scoped session substitution and unresolved-work review.

Acceptance tests: handover_no_overlapping_leader; incoming_master_verified; outgoing_access_revoked; history_author_preserved; queued_old_writes_quarantined; substitute_scope_expires.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

19 | Tasks H08-H11: reliable academy core

H08 | QR attendance and digital pass (original 2.4)

Depends on: H06/H01. Proposed module: passes/attendance.

Contract: getStudentPass(studentId); resolvePass(pass, sessionId); enqueueAttendance(mutation)

Deliver: Implement student/guardian pass and Master scanner with manual fallback, approved credential model, offline roster checks and preserved queue. Document expiry/revocation limits.

Acceptance tests: wrong_tenant_or_session_rejected; duplicate_scan_no_duplicate_mutation; camera_denial_manual_fallback; expired_credential_rejected; Android_iOS_camera_runtime; offline_50_student_recovery.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H09 | Persistent configurable belts (original 2.5)

Depends on: H02. Proposed module: curriculum.

Contract: publishBeltSystemVersion(tenantId, input); getCurriculum(versionId)

Deliver: Persist tenant-owned curriculum and admin editor, explicit Dan metadata, eligibility criteria and historical snapshots. Keep optional template import separate from authoritative configuration.

Acceptance tests: no_global_eight_rank_fallback; cross_tenant_rank_rejected; old_certificate_label_stable; invalid_rank_order_rejected; publish_version_preserves_existing_awards.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H10 | Scheduling and membership foundation

Depends on: H06. Proposed module: scheduling/membership.

Contract: createSession(input); applyScheduleException(input); getMembership(studentId)

Deliver: Add canonical sessions, recurring schedule exceptions, venue updates, clashes and membership status/expiry. Notify authorized families of changes. Financial renewal remains H13.

Acceptance tests: overlapping_resource_conflict; timezone_boundary_correct; cancelled_session_cannot_accept_new_attendance; membership_state_not_inferred_from_UI; schedule_notice_scoped.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H11 | Safety, notices and acknowledgements

Depends on: H05/H10. Proposed module: safety/notices.

Contract: publishNotice(scope, content, acknowledgementRequired); acknowledgeNotice(id); getSafetySummary(studentId)

Deliver: Ship restricted emergency/pickup data and consent foundation; basic operational notices and acknowledgement receipts. Use outbox retries and minimal notification payloads.

Acceptance tests: wrong_guardian_denied; sensitive_data_not_in_push; acknowledgement_not_consent; removed_member_no_notice_access; duplicate_delivery_no_duplicate_notice; safety_access_audited.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

20 | Tasks H12-H15: first-release completion

H12 | Grading, readiness and certificate verification

Depends on: H09/H06. Proposed module: grading/certificates.

Contract: getReadiness(studentId, rankVersionId); finalizeGrading(id, expectedVersion); verifyCertificate(token)

Deliver: Implement rubric-based assessments, readiness passport and authorized finalization; issue immutable certificate snapshot. Resolve CERT-001 using narrow public-safe verification and revocation.

Acceptance tests: attendance_alone_no_promotion; rubric_version_retained; unauthorized_finalize_rejected; public_verify_no_contact_or_dues; revoked_certificate_marked; guessed_tokens_rate_limited.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H13 | Payments, receipts and membership renewals

Depends on: H05/H10. Proposed module: billing/payments.

Contract: createPayment(billIds, idempotencyKey); processVerifiedProviderEvent(event); reconcilePayment(id)

Deliver: Select validated provider integration, server-owned Persatuan merchant routing, immutable ledger allocations, confirmed receipts, renewal event and controlled refunds. Include sandbox evidence and recovery after timeout.

Acceptance tests: merchant_amount_currency_mismatch_rejected; bad_signature_rejected; duplicate_webhook_once; late_success_reconciles; return_URL_not_payment_proof; refund_preserves_original_ledger.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H14 | Academy setup, imports and essential family view

Depends on: H03-H13. Proposed module: onboarding/imports/family.

Contract: previewImport(file); commitImport(previewId, idempotencyKey); getFamilySummary()

Deliver: Guide academy setup and validated legacy imports. Preview errors/duplicates and require authorized confirmation. Import into the same identities and approval model; expose approved children, next classes, notices and dues.

Acceptance tests: import_preview_no_mutation; import_retry_no_duplicate; unverified_parent_link_rejected; wrong_tenant_rows_rejected; family_checkout_separates_merchants; malformed_rows_reported.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H15 | Core certification and pilot release gate

Depends on: H08-H14. Proposed module: release-quality.

Contract: produce R1 evidence matrix and acceptance report

Deliver: Separate original 2.6 core attendance/belt certification from expanded R1 certification. Run full role journeys, data migration rehearsal, native devices, localization/accessibility, backups, payment sandbox and security boundaries. No public release while mandatory gaps remain.

Acceptance tests: Master_apply_to_student_approval_E2E; offline_QR_sync_E2E; grading_to_certificate_E2E; payment_to_receipt_E2E; cross_role_negative_matrix; restore_rehearsal_pass.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

21 | Tasks H16-H19: flexible class operations

H16 | Trials and actionable follow-up

Depends on: H05/H10/H11. Proposed module: trials/retention.

Contract: createTrial(input); convertTrial(id); evaluateFollowUpRules(classId); recordFollowUp(id, outcome)

Deliver: Add trial booking, consent and Master-approved conversion to canonical enrollment. Provide transparent absence, missed-trial and expiry follow-up queue with configurable rules and dismissed/resolved states.

Acceptance tests: trial_conversion_no_duplicate; conversion_requires_master; follow_up_only_own_class; reason_visible; corrected_attendance_recalculates_flag; no_auto_contact_without_authorized_action.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H17 | Make-up classes and waitlists

Depends on: H10/H05. Proposed module: bookings.

Contract: requestMakeUp(input); approveMakeUp(id, version); promoteWaitlist(sessionId)

Deliver: Validate class/program/level eligibility, membership rules and remaining places. Store session-only attendance entitlement without duplicating primary enrollment. Notify accepted families and release expired offers.

Acceptance tests: simultaneous_last_seat_one_booking; ineligible_student_rejected; cancelled_offer_releases_place; make_up_does_not_change_primary_class; no_cross_tenant_booking.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H18 | Pauses, returns and controlled transfers

Depends on: H10/H13/H07. Proposed module: membership/transfers.

Contract: requestPause(input); approvePause(id); resumeMembership(id); approveTransfer(id, expectedVersion)

Deliver: Apply versioned billing/expiry policy for leave, reminders and return. Implement within-Persatuan release/receiving approval and effective-date transfer. Add cross-Persatuan sharing only with explicit consent and receiving policy.

Acceptance tests: pause_no_silent_bill_erasure; return_restores_correct_status; transfer_preserves_student_id; concurrent_transfer_rejected; cross_tenant_no_implicit_history_copy; target_capacity_checked.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H19 | Lesson plans and family hub depth

Depends on: H06/H10/H12. Proposed module: teaching/family.

Contract: saveLessonPlan(sessionId, version, input); getSubstituteBrief(sessionId); getChildTimeline(studentId)

Deliver: Add objectives, drills, equipment, covered material and limited substitute briefing. Expand parent child-switching, progress and calendar summaries without merging tenant authorizations.

Acceptance tests: substitute_brief_hides_private_messages; lesson_edit_version_conflict; revoked_guardian_loses_access; long_multilingual_child_names_readable; family_totals_correct_per_merchant.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

22 | Tasks H20-H23: learning and engagement

H20 | Learning library, performance and rewards

Depends on: H09/H12/H19. Proposed module: learning/performance/rewards.

Contract: publishLesson(input); recordLessonProgress(input); recordAssessment(input); awardAchievement(eventId)

Deliver: Reuse syllabus skills and versioned rubrics; add approved learning content, quizzes, comparable progress charts and verified achievement events. Avoid global child leaderboards.

Acceptance tests: unpublished_lesson_hidden; cross_rubric_scores_not_falsely_compared; replay_event_one_award; revoked_award_audited; learning_completion_not_formal_grading.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H21 | Private video feedback

Depends on: H11/H20. Proposed module: media/feedback.

Contract: requestUpload(studentId, metadata); finalizeUpload(id); addVideoFeedback(id, input); revokeMediaConsent(id)

Deliver: Use consent-bound uploads, file validation, private object storage, signed access and authorized annotations. Apply retention/deletion jobs and remove access after consent/relationship changes.

Acceptance tests: oversized_invalid_upload_rejected; expired_URL_denied; wrong_Master_or_guardian_denied; consent_revocation_blocks_new_access; deleted_media_not_returned_by_API.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H22 | Parent-Master messaging

Depends on: H11/H07. Proposed module: messaging.

Contract: createThread(childId, classId); sendMessage(threadId, input); reportMessage(id)

Deliver: Add scoped threads, delivery/read states, reporting, moderation and retention. Enforce communication policy for minors; leadership replacement does not automatically expose old private threads.

Acceptance tests: cross_class_thread_denied; former_master_cannot_send; no_private_thread_auto_handover; message_retry_once; blocked_attachment_rejected; lock_screen_content_minimized.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H23 | Growth analytics and authorized exports

Depends on: H13/H16/H18. Proposed module: analytics.

Contract: getAcademyMetrics(scope, period); exportAuthorizedReport(query)

Deliver: Define denominators and metric versions for enrollment, trial conversion, retention, utilization and ledger totals. Restrict queries and exports to authorized aggregates or delegated records.

Acceptance tests: tenant_filter_enforced_server_side; finance_matches_ledger; transfers_not_double_counted; pending_applicants_not_active_students; export_cannot_expand_query_scope.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

23 | Tasks H24-H28: advanced modules and polish

H24 | Tournament operations and judging

Depends on: H12/H13. Proposed module: tournaments.

Contract: registerCompetitor(input); assignJudge(input); submitScore(matchId, version, score); lockResult(id)

Deliver: Extend existing events with ruleset, categories, brackets, judge scope, score locks, appeals and results. Rehearse connectivity fallback before live electronic judging.

Acceptance tests: judge_only_assigned_match; locked_score_requires_appeal; tie_rule_deterministic; duplicate_score_once; category_eligibility_checked.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H25 | Equipment and merchandise

Depends on: H13. Proposed module: commerce/inventory.

Contract: reserveStock(orderId); confirmOrderPayment(event); refundOrder(id)

Deliver: Add products, stock movements, orders, reservation expiry, refunds and scoped staff permissions using the shared ledger.

Acceptance tests: last_item_race_one_reservation; late_payment_no_negative_stock; refund_restock_policy_explicit; cross_tenant_order_denied.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H26 | Role-aware grounded AI

Depends on: H20/H23. Proposed module: assistant.

Contract: answerAuthorizedQuestion(actor, query, context) -> answerWithSources

Deliver: Retrieve only authorized approved sources, evaluate prompt injection and tenant isolation, cite evidence, bound cost and log minimally. Launch read-only assistance first.

Acceptance tests: retrieval_cross_tenant_denied; document_instruction_cannot_override_policy; unknown_answer_admits_gap; no_auto_grading_or_payment; Super_Admin_no_child_data.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

H27 | Advanced release certification

Depends on: H16-H26. Proposed module: release-quality.

Contract: produce R2/R3/R4 evidence independently

Deliver: Certify only enabled modules in each release; test recovery, privacy, load, media, messaging, scoring and commerce. Stage rollout with feature flags and rollback evidence.

Acceptance tests: disabled_module_inaccessible; migration_restore_pass; worker_replay_safe; release_specific_native_E2E_pass.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

23 continued | Usability across every release

H28 | Continuous localization and usability

Depends on: H00 onward; final pass before each release. Proposed module: design-system/localization.

Contract: localized role journeys and accessibility evidence

Deliver: Implement keys throughout, complete English/Malay/Tamil, test real users across roles, long strings, screen readers, low-memory devices and slow networks. Fold fixes into owning task.

Acceptance tests: Tamil_font_coverage; large_text_no_clipping; focus_order_usable; status_not_color_only; pending_approval_next_step_clear.

[ ] Map files and existing contracts. [ ] Add the named failing tests and confirm the intended failure. [ ] Implement the smallest compatible migration, server and mobile change. [ ] Run focused tests and relevant regression checks. [ ] Record evidence, migration/rollback notes and commit; stop at the task boundary.

Practical usability review

• Ask a new Master to apply, understand pending verification, propose a venue and identify what prevents the class from going live. The status screen should answer each question without staff explaining the system.

• Ask a parent to choose the right Persatuan and class, apply for a child and find the pending decision. Distinguish application submission from accepted enrollment.

• Ask a Master to approve a student, start attendance, lose connectivity and recover. The interface must show exactly what is local, synchronized or in conflict.

• Ask the Persatuan reviewer to compare a proposal with the submitted version and request corrections. Do not allow ambiguous approval of changed information.

• Ask a family with children in different classes to find the next lesson and pay the correct bill. Confirm the recipient Persatuan is visible and the payment is not marked successful before verification.

Keep advanced modules out of the way

Enable learning, video, tournaments, commerce and AI progressively. Keep the core navigation stable and place occasional actions under contextual menus or More. Use plain language rather than exposing database states, permission codes or provider integration details to families.

Performance checks

Measure launch, list rendering, camera start and local attendance-save latency on a representative lower-memory Android device and supported iPhone. Establish pilot budgets from measured devices, then prevent regressions. Use pagination, bounded roster caches and thumbnail media rather than loading entire histories or videos into memory.

24 | Feature coverage and acceptance gates

Original 14 additions	Owning tasks
1 Performance intelligence; 2 Video feedback	H20; H21
3 Tournament/judging; 4 Silambam learning	H24; H20
5 Smart scheduling; 6 Achievements	H10/H17; H20
7 Parent-Master communication; 8 Membership/renewals	H11/H22; H10/H13/H18
9 Growth analytics; 10 Safety/emergency	H23; H11
11 Trial enrollment; 12 English/Malay/Tamil	H16; H28 throughout
13 Equipment/merchandise; 14 AI assistant	H25; H26

New 10 additions	Owning tasks
A Master Today; B Follow-up queue	H06; H16
C Make-ups/capacity; D Pause/return	H17; H18
E Readiness passport; F Lesson plans/handover	H12; H19
G Family hub; H Notice acknowledgements	H14/H19; H11
I Controlled transfers; J Onboarding/imports	H18; H14

Core launch blockers

Do not launch R1 with unverified Master/class/student approval boundaries, cross-tenant exposure, lost or duplicated attendance, insecure QR behavior, inaccurate ledger/receipts, missing native runtime evidence, destructive migrations, unresolved public certificate mismatch or unusable critical role journeys.

Evidence format

Every gate records device/OS, environment, test command, actual result, commit, screenshots where useful and remaining limitations. Keep unit/component, SQLite, PostgreSQL, HTTP and native results distinct. A passing test count is not a substitute for workflow coverage. Use approved staging data, never sensitive real student records in screenshots or logs.

Pilot acceptance

Pilot with authorized representatives of Persatuan administration, Masters, parents and students as appropriate. Observe registration, approval, attendance, class change, payment and progression tasks. Record failures and revise UX before expanding. Dates, staffing and estimates follow H00 repository sizing; there is no supported fixed delivery promise yet.

25 | Deployment, recovery and operational ownership

Environments and configuration

Separate development, test/staging and production databases, provider credentials, media storage and notification settings. Keep secrets in server-side secret management, not source code or app bundles. Validate environment configuration at startup. Mobile builds use environment-appropriate endpoints and cannot choose arbitrary tenant merchants.

Data and deployment sequence

Back up -> deploy compatible additive schema -> deploy backward-compatible server -> validate backfill in a controlled rehearsal -> release mobile build -> monitor adoption/errors -> enable feature by tenant -> remove obsolete paths only after evidence and approval. Mobile clients may remain old; define supported API/client versions and a safe update message.

Rollback and recovery

Use feature disablement and backward-compatible code rollback first. Do not reverse a migration by dropping newly created real records. For data faults, pause affected writes, preserve evidence, use a rehearsed restore/forward repair and reconcile provider events after recovery. Test crash windows in sync, payment, stock and handover workers.

Monitoring

• Track approval failures, queue age, sync conflicts, idempotency collisions, stale-version errors, notification backlog and payment reconciliation mismatches.

• Alert on cross-tenant authorization failures, failed webhook verification, unexpected leadership changes and abnormal verification attempts without logging sensitive payloads.

• Expose support diagnostics such as app version and operation reference. Redact JWTs, personal identifiers, safety details and merchant secrets.

• Assign owners for incident response, merchant reconciliation, consent/retention, curriculum publishing, app-store release and tenant onboarding before production.

Retention and exceptional governance decisions

Record actual policy for application documents, declined applicants, local cache expiry, media, messages, audit retention and account deletion. Specify exceptional suspension/reinstatement authority and appeal handling. These are bounded activation prerequisites for their modules, not reasons to invent permissions or block unrelated foundational tasks.

Definition of done

Compatible code and migrations; correct authorization; complete loading/error/offline states; localization keys; meaningful passing tests; actual native evidence when relevant; clear operational recovery; reviewed changes; commit and task report. Report not-tested conditions explicitly. Do not automatically merge, publish or proceed into production.

26 | Copy-ready Antigravity handoff

Start with this instruction

Continue the existing BeltFlow repository using this PDF as the current product specification and staged implementation roadmap. Read Sections 1-26 and consult Appendix A for historical context. Latest confirmed workflow: Persatuan verifies the Master; verified Master proposes the class; Persatuan approves the class before operation; students choose an approved available class under that Persatuan; its current class Master approves enrollment. The Master operates the class. Persatuan-approved permanent handover replaces its Master while preserving the class and all history.

Do not restart the project, replace the backend, assume missing features are absent, or treat historical test reports as independently verified. Preserve the existing Android/iOS mobile code, Express/PostgreSQL backend, offline attendance and legacy reference assets. Use the installed Superpowers workflow and follow applicable repository instructions.

First execution boundary: H00 only

Inspect the repository and return: current branch and working-tree state; verified baseline commit and actual tests; current schema/routes and exact module file paths; the present ownership/approval behavior; a gap matrix against this PDF; proposed migration and compatibility approach; actual commands for validation; blockers and decisions that genuinely require the owner. Do not modify production data or silently approve legacy records.

Then prepare the exact file-level implementation slice for H01/H02 based on those findings. If the real code already satisfies a requirement, prove it and reuse it. If a proposed interface conflicts with an existing sound contract, explain the compatibility adaptation. Continue only within the task scope authorized by the user; do not execute all H00-H28 as an unattended batch.

Task completion report template

- Task ID and result: completed / partial / blocked, with user-visible behavior.
- Branch, baseline and commit SHA; files created/modified; migrations and contract changes.
- Permissions and state transitions implemented; preserved legacy behavior.
- Exact validation commands and real outcomes, separated by unit, database, HTTP and native tests.
- Screenshots or device evidence for relevant user journeys; do not expose student data.
- Known defects, unverified assumptions, rollback/recovery notes and pending business decisions.
- Next recommended task with dependency status; no automatic production publish.

First review questions to answer from the code

Does the backend already model Master ownership independently of assignment? Can unapproved classes or students become active? Are students duplicated when moving classes? Are parent links verified? Is leadership revocation rechecked during offline sync? Does class control accidentally grant merchant or Persatuan-wide access? Report evidence for each.

Appendix A | Original project history

The following 14 pages reproduce the supplied source PDF unchanged, preserving the full prior roadmap, reported commits, tests, defects and supporting report references.

Source filename: BeltFlow_Complete_Project_History_and_Roadmap_A_to_Z (2).pdf. Source report date: 15 October 2026.

How to interpret conflicts

The new plan preceding this appendix supersedes older implications that Persatuan normally assigns classes to Masters. Assigned-class access remains a useful technical authorization concept, but the business creation flow is Master proposal followed by Persatuan approval.

The original six-phase roadmap and all 14 additions remain included. This document adds all 10 approved ideas and explicit Master verification, class approval, student application and permanent leadership handover workflows. It moves essential notices, payments, grading and native testing into the first-release requirements rather than waiting for the end of their historical phase labels.

No original implementation claim is upgraded to verified status by inclusion in this appendix. Source-document instructions are historical context; the copy-ready handoff above defines the requested next step.

BELTFLOW

Complete Project History, Implementation Reports, Architecture, Decisions and Six-Phase Future Roadmap (A-Z)

Report date: 15 October 2026 Scope: All BeltFlow discussions and Antigravity completion reports shared in this conversation.

Executive position

BeltFlow began as a multi-Persatuan academy management system. Its scope was deliberately changed from web plus Android to one Android/iOS mobile app with five server-controlled roles, an existing Express/PostgreSQL backend, offline attendance, and FPX/DuitNow payments paid directly to each Persatuan merchant account. Phase 1 reached a Conditional Pass; Phase 2 tasks 2.1, 2.2 and 2.3 were reported complete. Task 2.4 (QR) remains next. A six-phase extended roadmap now contains 14 proposed new features.

Evidence distinction: Design decisions and roadmap choices were made within the chat. Implementation, tests, commits, and certifications below are reported by Antigravity in user-shared documents; this PDF does not independently inspect the Git repository or run software. A passing Metro bundle is not proof of a functioning native app.

Contents

01 Origin and confirmed business decisions

02 Technical architecture and existing legacy assets

03 Five roles and access model

04 Phase 1 chronological completion record

05 Phase 2 readiness gate and completed tasks

06 Attendance, QR, payment and belt design rules

07 Expanded six-phase roadmap and 14 additions

08 Features, dependencies, permissions and data plans

09 Outstanding defects, risks and testing gates

10 Development workflow, Git and next actions

11 Consolidated record of milestones and source files

01 | Where BeltFlow started

Initial product scope and decisions

- Initial proposal included a web application and Android application, with one PostgreSQL backend and role-based workflows.
- Web app was explicitly removed. Final customer-facing target: one shared app for Android and iOS. The existing web prototype remains temporarily as a reference during migration, not a target product.
- One app serves all five roles, with screens and permissions selected from the authenticated backend role.
- Masters must take attendance without internet, retain local changes, and synchronize automatically when connectivity returns.
- First mobile release must support FPX and DuitNow through an approved Malaysian payment provider. Each Persatuan receives money directly into its own merchant account. BeltFlow does not act as a centralized settlement collector.
- Digital receipts and verified payment callbacks are required; a mobile return URL is not sufficient evidence that payment succeeded.
- Belt grading rules differ among Persatuans. Names, number, colors, order, Dan titles, fees, eligibility criteria and certificates are not globally fixed.

Five confirmed roles

Role	Primary access	Critical restriction
SUPER_ADMIN	Organization provisioning, platform status, global audit	No student, fee, attendance or grading operational records
ADMIN_PERSATUAN	Persatuan settings, locations, Masters, authorized reports, belt configuration	Cannot assume Master-only operational privileges
MASTER	Assigned classes, roster, sessions, attendance, grading, authorized fee handling	Only assigned classes and delegated workflows
PARENT	Approved children, attendance, fees, notices, passes, certificates	Only validated parent-child links
STUDENT	Own schedule, attendance, skills, belts, certificates, notices	Own records only

Scope boundaries

BeltFlow is a multi-tenant system: Persatuan A must never see or alter Persatuan B records. Browser-facing PWA functionality was retained only as migration reference. Later capabilities can add scoped judge or inventory permissions without introducing new global roles by default.

02 | Architecture discovered in the existing project

Layer	Reported existing system	Migration approach
Backend	Express + TypeScript; REST /api/v1; PostgreSQL via pg.Pool or PGlite	Reuse server and database contracts
Original Android	Kotlin, Jetpack Compose, Material 3, Room; 17 screens	Keep as behavior/design reference; not cross-platform
Web prototype	Single index.html, ~6,373 lines of CSS/JS and UX	Preserve as reference; not a customer-facing release
Mobile replacement	React Native + Expo + TypeScript	One codebase, Android and iOS
Secure storage	Expo SecureStore for JWT/session	No JWTs in unencrypted SQLite
Offline store	Expo SQLite, migrations, queue, conflict records	Authorized roster caching and durable attendance operations
UI/UX	Navy/amber brand, theme provider, 5 navigators	Accessible, role-aware components

Original belt branding identified in the audit: primary navy #0D2B47, dark navy #061828, light navy #153C61, accent amber #F2941C; typography Inter and Space Grotesk. Antigravity reported the Phase 1 mobile project using Expo SDK 57, React Native 0.86, React 19, and TypeScript.

Existing backend domains identified at first audit

Authentication; organizations; classes; students; attendance; billing; grading; certificates; coaches; tournaments; skills; notices; and audit. The audit described 13 API route groups and an established multi-tenant relational model. Some later requirements need additive tables and endpoints, not another parallel database.

03 | Phase 1: complete chronological task record

Task	Reported delivery	Mobile tests	Commit
1.1	Expo TypeScript mobile workspace, package IDs, basic tests	5 / 5	0ce67b0
1.2	Design tokens, themes, typography, belt-color helper	22 / 22	9fcfcd9
1.2 fix	Persatuan-customizable belts; eight-rank default is optional	32 / 32	df3b202
1.3	Buttons, Cards, Inputs, Badges, Stats, Feedback, Layout	54 / 54	8031e0d
1.3 fix	Real ESLint; Dan only when metadata explicitly declares it	Carried forward	827b044
1.4	Typed API client, SecureStore, 7-day JWT session management	73 / 73	416024b
1.5	Splash, Welcome, Login, Recovery info, Certificate verify UI	92 / 92	34afe60
1.6	Role-isolated tabs/stacks, deep-link barriers, online session gate	125 / 125	3343586
1.7	Five read-only role dashboards and typed data hooks	139 / 139	628ad79
1.8	Connectivity, backend health, resilience UI, sync state contract	158 / 158	8bd7454
1.9	Security audit, extra server endpoint tests, certification	158 / 158	64d7b1c

Phase 1 user experience

- A role-specific app shell mounts only the authenticated role navigator. Login, logout, cached session display and online verification are provided.
- Public certificate lookup was exposed from the welcome screen, but a later real-HTTP readiness audit found its API requires authentication. This is open defect CERT-001.
- Five dashboard shells show appropriate data, read-only summaries, loading/error/empty states and navigation to later workflow placeholders.
- Theming supports light/dark display; belts are dynamic tenant data. When no curriculum exists, no false "highest rank" claim is made.
- Offline UI differentiates device connectivity from backend reachability and does not claim there are synchronized attendance records when no real sync engine exists.

Phase 1 certification outcome

Conditional Pass on 14 October 2026, tagged v0.1.0-phase1 at commit 64d7b1c. Twelve mobile suites/158 tests were reported passing; server RBAC tests, typecheck, ESLint, Android and iOS Metro exports also passed. Actual Android/iOS app launches were not verified. The legacy Android app, server and PWA reference were reported preserved.

04 | Phase 2: readiness gate and completed tasks

Phase 2 readiness gate

Branch feat/phase2-native-and-integration was created from the Phase 1 tag. Antigravity reported 61/61 live HTTP assertions against a controlled Express/PGlite setup spanning authentication, dashboards, privacy boundaries and tenant permissions, plus 158 passing mobile tests. Commit 5c55c55 added a backend readiness suite. Android SDK/ADB/AVD were absent on Windows; iOS Xcode/Simulator also unavailable. This gate exposed CERT-001: unauthenticated certificate verification does not match the protected backend route.

Task 2.1 | SQLite foundation

Reported commit a953b27; 173/173 mobile tests and 15 real SQLite integration scenarios. Added Expo SQLite adapter and a Node test adapter, migrations, session/roster/queue/conflict repositories, foreign keys, and strict user/tenant/class access helpers.

- Tables: schema_migrations, cached_classes, cached_training_groups, cached_students, cached_training_sessions, attendance_mutation_queue, sync_conflicts and sync_metadata.
- Mutation identity and correction sequence modeled separately; retries preserve durable IDs.
- Data minimization excludes national IDs, dates of birth, banking credentials and payment details from student cache.
- Safety caveat: sandbox storage is not a substitute for full database encryption; backup exclusions and sensitive-data retention require platform verification.

Task 2.2 | Master attendance while offline

Reported commit 02b6a90; 191/191 tests. Created an attendance screen: assigned class/group, scheduled session, roster search, historical/custom belt badges, PRESENT/LATE/ABSENT/EXCUSED controls, summary counts, local save, pending-sync badge, corrections and failure feedback.

- Cache scrubbing on account switch, including a new Master within the same Persatuan.
- Unsent records are quarantined on logout, account change, expired authorization or revoked class assignment rather than silently deleted.
- Need persistent device-security review and verified recovery of unsaved/queued changes.

Task 2.3 | Automatic offline synchronization

Reported commit 94f82ed; 216/216 mobile tests across 15 suites and a passing PostgreSQL-backed server sync integration suite. Added POST /api/v1/attendance/sync, processed_sync_mutations, attendance version/updated_at, and an offline sync worker.

- Batch limit 25, reconnection/foreground/manual triggers, retry/backoff with jitter, mutex, crash recovery and actual SQLite pending/conflict counts.
- Processed mutation IDs protect replay handling; attendance changes and conflicting edits use server-authoritative validation.
- Conflicts include closed session, transfer, inactive student, revoked Master, concurrent edits, bad status and cross-tenant attempts.
- Important follow-up: independently verify transaction atomicity (attendance update plus idempotency ledger), race-safe optimistic locking, and reauthorization before replay.

Where the project stands

Task 2.4 QR Attendance Scanner & Student Digital Pass: a development prompt was prepared, but no task 2.4 completion report has been provided. Tasks 2.5 (persisted custom belt configuration) and 2.6 (integration/certification) remain planned.

05 | Critical architectural decisions and rules

Offline attendance lifecycle

Authorized Master downloads only an assigned roster/session while online; attendance is saved to protected local SQLite while disconnected; UI displays Pending Sync; reconnection triggers server validation, persistence, idempotent acknowledgement, or explicit conflict/quarantine. Offline save is not the same as server sync. App termination may suspend background work; foreground recovery is necessary.

QR pass security - Task 2.4 planned

- Student digital QR pass and parent access to approved linked children; authorized Master scanner, flash and manual fallback.
- QR payload must avoid NRIC/MyKid, address, phone, payment data and bearer JWT. Prefer opaque nonenumerable or signed credentials.
- Require authorized session and roster validation, cross-Persatuan/class protection, duplicate-scan handling, and local offline save with later server verification.
- Offline revocation cannot always be checked in real time; present limits honestly. A QR code alone does not establish the presenter is the genuine student.
- Real Android and iOS camera testing is required; bundle exports and mocked camera tests are insufficient.

Configurable grading and historic integrity

Every Persatuan independently controls belt names, count, sequence, colors, instructor/Dan metadata, fees, minimum sessions/months, assessment rules and certificate display. The eight-rank White-to-Black template is optional, never a global authority. Existing historical labels and certificates must remain intact. The currently reported in-memory domain model requires persistent PostgreSQL configuration and admin editing as a future task.

Financial and merchant architecture

Parents should be able to pay via approved Malaysian FPX/DuitNow rails. Each Persatuan has its own merchant account and receives funds directly. The backend must verify payment gateway callbacks before updating balances or issuing verified receipts. Existing proof-upload/manual DuitNow flows must not be presented as identical to real-time gateway settlement. Reuse one auditable ledger across tuition, grading, renewal, events and merchandise.

Role-appropriate privacy

The five top-level roles remain constant. Tournament judges and inventory helpers should generally use scoped role permissions instead of platform-wide new user roles. Minors, parent-child relationships, videos, emergency records, and messaging all need consent, access logs, retention and deletion processes. Security checks belong on the server, not merely in UI filters.

06 | Original six-phase roadmap, expanded to BeltFlow 2.0

Phase	Status	Included scope
1 - Foundation	Conditional Pass	Android/iOS shell, theme, login, five navigators and dashboard shells, security and offline UI
2 - Academy Operations	In progress	Offline/QR attendance, student/class management, schedules, trial enrollment, memberships, safety foundation, per-Persatuan belt persistence, language infrastructure
3 - Grading & Learning	Planned	Belt criteria, assessments, certificates; student performance intelligence, learning library, video feedback, achievement rewards
4 - Finance & Commerce	Planned	FPX/DuitNow direct merchant accounts, billing and receipts, renewals, equipment, merchandise and inventory
5 - Tournaments & Engagement	Planned	Tournament brackets and electronic judging, parent-master messaging, notifications, growth analytics and reports
6 - AI & Release	Planned	Role-aware BeltFlow AI, full English/Malay/Tamil coverage, native testing, security/privacy certification, store release

Proposed delivery distinction

Essential first public release: secure mobile login, students, classes, active membership, scheduling, offline and QR attendance, Persatuan grading, confirmed payments/receipts, certificates, baseline communication and safety, plus mandatory native/security testing. Advanced release: learning video, sophisticated performance analytics, electronic competition judging, merchandise, rewards, AI and deeper analytics. All six phases remain the planning framework; separate releases can happen within that framework after acceptance gates.

07 | Fourteen proposed additions (no duplicate core systems)

01. Performance Intelligence [P3]

Use existing assessment, attendance and grading entities; scored criteria such as Power, Speed, Style, Skill, Variety. Primary roles: Master enters; student/parent views.

02. Video Training Feedback [P3]

Student video uploads, technique annotations, feedback, consent and retention. Primary roles: Master, student, parent with consent.

03. Advanced Tournament/Judging [P5]

Extend tournament roster with categories, matches, judge assignments, scorecards, appeals and medals. Primary roles: Admin, Master, scoped judge.

04. Digital Silambam Learning [P3]

Persatuan-owned curriculum, videos, quizzes and progress, reusing syllabus skills. Primary roles: Master authors; student learns.

05. Smart Class Scheduling [P2]

Exceptions, venue changes, substitute Masters, collision checks and alerts. Primary roles: Master/Admin; family views.

06. Achievements & Rewards [P3]

Badges linked to real attendance, grading, learning and events. Primary roles: Master awards; family views.

07. Parent-Master Communication [P5]

Private conversations, messages, absence reasons, meeting requests and moderation. Primary roles: Parent, assigned Master.

08. Digital Membership/Renewals [P2/P4]

Membership lifecycle and expiry; renewal payments reuse existing verified ledger. Primary roles: Admin; student/parent.

09. Academy Growth Analytics [P5]

Authorized enrollment, fee, retention and class utilization aggregates. Primary roles: Admin, scoped Master.

10. Safety/Emergency Records [P2]

Emergency contacts, incident logs, pickup consent, health alerts with extra controls. Primary roles: Authorized Admin/Master/Parent.

11. Trial Enrollment [P2]

Inquiry, trial scheduling, parent consent, conversion to existing student record. Primary roles: Admin/Master/Parent.

12. English/Malay/Tamil [P2/P6]

Localization keys and dynamic content translation strategy. Primary roles: All roles.

13. Equipment & Merchandise [P4]

Products, stock transactions, inventory, orders, refunds with existing payment engine. Primary roles: Admin, scoped staff, buyers.

14. AI Assistant [P6]

Knowledge-grounded role-specific assistance, tenant permissions, audit, usage limits. Primary roles: All five roles, scoped.

08 | Dependencies and implementation order

Prerequisite	Unlocks	Reason
Auth, RBAC, organizations	Every new feature	Tenant and person-scoped access is foundational
Students, classes, groups	Trials, memberships, scheduling, QR	Single canonical student and class records
Attendance + grading	Performance and achievements	Derived outcomes use verified source records
Syllabus and learning	Video feedback and AI assistance	Trusted curriculum materials before advisory features
Fee ledger and gateway verification	Renewals, merchandise, event fees	One reconciled financial source of truth
Safety and media consent	Student videos, event publishing, messaging	Minor/student protection requirements
Tournament master records	Judging, medals, analytics	Avoid duplicate competition histories
Authorized report datasets	Growth analytics and AI	Prevent oversharing and misleading insight

Suggested task order after current baseline

2.4: QR Scanner and Student Pass; 2.5: Persistent Persatuan Belt Configuration; 2.6: Phase 2 operational certification. Then Phase 2 extensions can be planned as independently versioned tasks for scheduling, trials, membership foundation, safety, and localization foundation, with a scope and date review before implementation. Phase 3 starts grading/development modules; Phase 4 handles finances; Phase 5 communications/competition/analytics; Phase 6 AI and release. No exact deadlines or delivery promises were established in chat.

Architecture-first guardrails

- Treat in-memory configuration as a prototype until backed by tenant-specific PostgreSQL records and authorized admin editing.
- Preserve canonical domain records and reference them rather than copying whole student, fee or grading histories into new modules.
- Keep read and write privileges distinct and use scoped assignments for judges, inventory staff and substitute Masters.
- Use database transactions and idempotency for billing, sync, inventory and electronic judging.
- Include bilingual/trilingual UI expansion and device testing in acceptance criteria rather than retrofitting at final release.

09 | Proposed database and API extensions (planning only)

Area	Potential additive schema	API need
Student performance	assessment_templates, student_assessments, assessment_scores	Assessments and authorized progress
Learning/video	learning_modules, lessons, student_lesson_progress, training_videos, video_feedback, media_consents	Upload/review/access-control endpoints
Scheduling	schedule_exceptions, class_reschedules, master_substitutions	Clash checks and updates
Achievements	achievement_definitions, student_achievements	Award/list/revoke audit
Messaging	conversation_threads, messages, message_receipts	Authorized send/read/report
Membership/trials	membership_plans, student_memberships, renewal_events, enrollment_applications, trial_sessions	Trial conversion and renewal
Safety/consent	emergency_contacts, student_safety_profiles, incident_reports, consent_records	Restricted access and retention
Commerce	products, stock_movements, orders, order_items	Stock and orders linked to ledger
Tournament judging	matches, judging_assignments, scorecards, result_appeals	Scoring, locks, dispute resolution
Analytics/AI	Aggregated views; ai_knowledge_sources, ai_audit_events	Scoped analytics and grounded AI

Important: These are proposed table names, not deployed changes or a final schema. Antigravity must inventory the actual database and migration history to avoid overlap. Existing grading and tournament tables should be extended, not duplicated.

10 | Quality gates, blockers and known gaps

Outstanding defects and validation gaps

ID	Status	Meaning / required evidence
CERT-001	OPEN	Mobile unauthenticated verification uses a route that currently demands JWT. Design a narrowly scoped, rate-limited public-safe verification contract.
NATIVE-001	OPEN	Android app actually launched and tested on physical hardware or emulator; camera and offline flow require hardware validation.
NATIVE-002	OPEN	iOS app launch on iPhone or simulator/remote macOS required; Metro export is insufficient.
QR runtime	FUTURE	Task 2.4 scanner must test camera permissions, scans and expiry/revocation scenarios on devices.
Sync atomicity	VERIFY	Confirm processed_sync_mutations insert and attendance UPSERT are transactional; test simultaneous stale versions and crash windows.
Privacy & retention	DESIGN	Check iOS/Android backups, deletion/retention, child media consent, local SQLite exposure and account switch boundaries.
Belt persistence	PLANNED	Create tenant-owned persistent belt systems and historical snapshot rules, not silent eight-rank fallback.

Testing matrix to retain

- React Native unit and component tests, real SQLite integration tests, PostgreSQL integration and HTTP route contract tests.
- Cross-tenant access checks for organizations, students, rosters, dues, training sessions, media, communication and AI.
- Attendance: offline 50-student test, interrupted upload, idempotency replay, conflicting edits, revoked assignment and quarantine.
- Payments: approved gateway sandbox, verified webhook state, retries, duplicate callbacks and refunds; do not certify using manual proof uploads alone.
- Performance: low-memory/offline behavior, usability/accessibility, dynamic font sizes, long translated strings and safe areas.
- Real Android device and actual iOS device/simulator checks before production release; camera and push notifications require extra hardware testing.

Privacy/safety and AI limits

Student safety and sensitive data must be restricted to authorized users with explicit consent and audit. AI responses should be grounded in approved resources, obey tenant and role boundaries, avoid inventing grading/financial decisions, and require human approval for sensitive actions. Super Admin should have platform administration only, not an unrestricted view of academy children.

11 | Working method: Superpowers + Antigravity

The user chose to work in Google Antigravity with a locally available existing BeltFlow repository. The installed Superpowers methodology was used as a planning/TDD discipline. The assistant supplied staged prompts; Antigravity returned task reports and commit hashes. The approach throughout: inspect existing code, plan a narrowly scoped task, write tests first where suitable, implement minimally, execute real checks, commit, report blockers, and stop before the next task.

Standard task completion contract

- Git branch and baseline confirmed; no destructive rewrite of the server, old Android reference, or PWA prototype.
- Files created/modified, API/database migration inventory and all new permissions specified.
- Tests with real output: typecheck, ESLint, Jest, SQLite/server, native exports and runtime status separately.
- Commit SHA, outstanding defects, unverified claims and readiness for next task documented.
- Do not automatically merge, publish, alter production data, or start a later task without instruction.

Immediate next steps

1. Review the Task 2.3 server transaction/concurrency details and preserve the reported 216-test baseline.

2. Implement Task 2.4 QR scanning with signed/opaque passes, offline roster validation and parent/student visibility.

3. Validate camera behavior on Android and iOS hardware; report unavailable runtime paths honestly.

4. Implement Task 2.5 PostgreSQL belt configuration with strong tenant consistency and historical versioning.

5. Finish Task 2.6 end-to-end Phase 2 certification, including public certificate contract fix or a clearly tracked blocker.

6. Ask Antigravity for the six-phase master planning report to reconcile all 14 additions with existing modules, and only then expand the backlog.

12 | Consolidated milestones and supporting reports

Date / milestone	Evidence file / record	Key result
Initial audit	Pasted text(4).txt	Existing server, Android/PWA inventory; mobile migration plan
Phase 1, task 1.5	Pasted text(5).txt	92 tests; authentication UI
Phase 1, task 1.6	Pasted text(6).txt	125 tests; role navigation
Phase 1, task 1.8	Pasted text(9).txt	158 tests; offline feedback and safety fixes
Phase 1 certification	Pasted text(10).txt	Conditional Pass; tag v0.1.0-phase1
Phase 2 readiness	Pasted text(20261014-083226).txt	61 live HTTP checks; native test blockers
Phase 2, task 2.1	Pasted text(20261014-085205).txt	173 tests; SQLite foundation
Phase 2, task 2.2	Pasted text(20261014-095009).txt	191 tests; Master attendance local queue
Phase 2, task 2.3	Pasted text(20261015-023817).txt	216 tests; offline sync and PostgreSQL protection

Final project status as of report date

Implemented / reported: Phase 1 foundation and role dashboards; Phase 2 offline SQLite, Master attendance, and synchronization through Task 2.3. Next: QR/student pass Task 2.4 and persisted belt configuration Task 2.5. Planned: Phase 2 certification and the expanded six-phase feature backlog. Open: public certificate verification mismatch, real Android/iOS runtime sign-off and other technical verification checks.

Document intent: This is a comprehensive, chronologically organized project record, not a verbatim chat transcript. It includes the substantial decisions, task summaries, reported test counts/commits, technical dependencies, risks and planned work from this conversation.